Project Sentinel

Initial Project Scope (Version 1.0)

Overview

Project Sentinel is an open-source autonomous telemetry and system health monitoring framework for Linux-based embedded systems.
The goal of the project is to develop a lightweight, modular platform capable of continuously monitoring system health, recording historical telemetry, analysing long-term performance trends, and providing early warning of potential hardware or software failures.
Rather than acting as a traditional system monitor that reports current system status, Sentinel focuses on identifying degradation over time, enabling predictive maintenance and improving the reliability of mission-critical computing systems.
The project will adopt a hybrid architecture consisting of a small kernel component responsible for low-level telemetry collection and a user-space daemon responsible for analysis, storage, alerting and remote communication.

Motivation

Modern embedded Linux systems are increasingly deployed in environments where reliability is critical. Examples include:

- Autonomous robots
- UAVs
- Aircraft avionics
- Satellites
- Spacecraft
- Industrial automation
- Self-driving vehicles
- Maritime navigation systems
- Mining equipment
- Medical devices
- Remote edge computing
- Research equipment

(Avoiding usage in Defence, this would be exclusively used in civilian applications)
While Linux provides excellent monitoring interfaces, there is currently no lightweight open-source framework focused on autonomous health monitoring and long-term system diagnostics.
Sentinel aims to bridge this gap.

Project Goals

The primary objectives are:

- Continuously monitor system health with minimal overhead
- Store long-term telemetry for historical analysis
- Identify performance degradation
- Identify early warning signs of hardware failure
- Allow remote monitoring and alerting
- Operate autonomously without user intervention
- Be portable across Linux-based systems
- Maintain high reliability and fault tolerance
- Be suitable for embedded environments
- Support secure remote telemetry

High-level Architecture



Core Features

Hardware Monitoring

Monitor:

- CPU utilisation
- CPU frequency
- CPU temperature
- GPU utilisation
- GPU temperature
- Memory usage
- Swap usage
- Disk utilisation
- SSD/HDD SMART data
- Disk temperatures
- Battery health (where available)
- Power consumption
- Voltage levels
- Fan speeds
- Thermal throttling events

Network Monitoring

Collect:

- Interface throughput
- Packet loss
- Connection failures
- Interface state changes
- Latency
- Bandwidth usage
- Error counts
- Wireless signal quality
- Cellular signal quality (optional)

Operating System Monitoring

Track:

- Process creation
- Process crashes
- Kernel panics
- OOM Killer events
- Kernel logs
- Service failures
- Boot history
- Uptime
- Unexpected shutdowns

Application Monitoring

Monitor:

- CPU usage per application
- Memory usage
- Thread count
- Crash frequency
- Runtime duration
- Resource leaks
- Restart frequency

Historical Data Collection

Store telemetry over long periods.

Examples:

- Last hour
- Last day
- Last week
- Last month
- Entire lifetime of the system

Historical data enables trend analysis rather than relying only on current values.

Trend Analysis

Automatically detect:

- Increasing temperatures
- Memory leaks
- Storage degradation
- Increasing boot time
- Declining network performance
- Battery degradation
- Increasing CPU utilisation
- Performance regressions after software updates

Anomaly Detection

Identify unusual behaviour such as:

- CPU suddenly running hotter
- Fan stopped responding
- Unexpected increase in memory usage
- New process consuming excessive resources
- Network traffic spikes
- Storage write amplification
- Thermal throttling
- Sensor failures

Initially these can be rule-based, with the possibility of machine learning later.

Alerting

Generate alerts for:

- High temperatures
- Critical temperatures
- Low battery
- Disk nearing failure
- Memory exhaustion
- Network disconnect
- Application repeatedly crashing
- Power instability
- Hardware degradation

Remote Telemetry

Securely transmit telemetry to external systems.

Potential protocols:

- MQTT
- HTTPS REST
- WebSocket
- gRPC

Possible features:

- Live dashboard
- Fleet monitoring
- Remote diagnostics
- OTA status updates

Logging

Persistent logging of:

- Alerts
- System events
- Hardware faults
- Software faults
- Sensor readings
- Network events

Non-Functional Requirements

Performance:

- Minimal CPU overhead (<2%)
- Minimal memory footprint
- Low disk usage
- Suitable for Raspberry Pi-class devices

Reliability:

- Runs continuously
- Automatic recovery
- Crash resistant
- Corruption-resistant logging

Security:

- Encrypted telemetry
- Authentication
- Secure configuration
- Least privilege where possible

Portability:

- ARM
- ARM64
- x86
- x86_64
- Embedded Linux distributions

Proposed Technology Stack

Component	Technology
Kernel Module	C
User-space Daemon	C
Build System	CMake
Testing	CTest + GitHub Actions
Documentation	Markdown + Doxygen
Remote API	REST / MQTT
Configuration	TOML or YAML

Out of Scope

The following features are intentionally excluded from this project, at least initially:

- Machine learning
- Cloud infrastructure
- Windows support
- macOS support
- GUI desktop application
- Mobile applications